Edge-Assisted On-Device Model Update for Video Analytics in Adverse Environments

Yuxin Kong
Huazhong University of Science and Technology, Wuhan, China
yxkong@hust.edu.cn

Peng Yang*
Huazhong University of Science and Technology, Wuhan, China
yangpeng@hust.edu.cn

Yan Cheng
Huazhong University of Science and Technology, Wuhan, China
y_cheng@hust.edu.cn

ABSTRACT

While large deep neural networks excel at general video analytics tasks, the significant demand on computing capacity makes them infeasible for real-time inference on resource-constrained end cameras. In this paper, we propose an edge-assisted framework that continuously updates the lightweight model deployed on the end cameras to achieve accurate predictions in adverse environments. This framework consists of three modules, namely, a key frame extractor, a trigger controller, and a retraining manager. The low-cost key frame extractor obtains frames that can best represent the current environment. Those frames are then transmitted and buffered as the retraining data for model update at the edge server. Once the trigger controller detects a significant accuracy drop in the selected frames, the retraining manager outputs the optimal retraining configuration balancing the accuracy and time cost. We prototype our system on two end devices of different computing capacities with one edge server. The results demonstrate that our approach significantly improves accuracy across all tested adverse environment scenarios (up to 24%) and reduces more than 50% of the retraining time compared to existing benchmarks.

CCS CONCEPTS

• **Information systems** → **Multimedia information systems**.

KEYWORDS

Video analytics; edge computing; neural networks; model update

ACM Reference Format:
Yuxin Kong, Peng Yang, and Yan Cheng. 2023. Edge-Assisted On-Device Model Update for Video Analytics in Adverse Environments. In *Proceedings of the 31st ACM International Conference on Multimedia (MM '23), October 29-November 3, 2023, Ottawa, ON, Canada.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3581783.3612585

1 INTRODUCTION

Empowered by the advancement of deep neural networks (DNNs) and the increasing prevalence of cameras, video analytics has become an indispensable component in various applications, such as traffic monitoring [17, 18, 37, 48], autonomous driving [1, 7, 14, 25], intelligent industry [8, 16], etc. While advanced DNNs can generate accurate inference results for various challenging analytics tasks [6, 12], the sophisticated network structures and substantial parameters result in a significant computation burden that renders such models impractical for real-time analytics on resource-constrained end devices [3, 5], e.g., most of the off-the-shelf cameras.

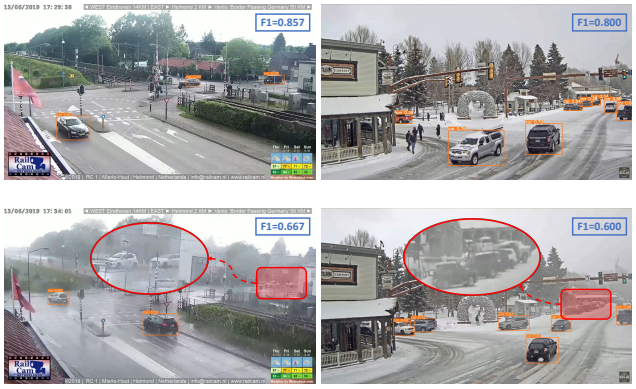


Figure 1: A comparison of lightweight Yolov5-n [53] performing object detection under normal environments (Row 1) and adverse environmental conditions (Row 2).

One common approach to tackle this challenge is to employ carefully pruned or compressed models that can adapt to the restricted computing resources of end devices, which has been proven effective in well-conditioned environments or under customized scenes [11, 19, 41, 58]. Nevertheless, it is difficult for lightweight models to achieve high-performance inference results in diverse scenarios due to the intrinsic simplified architecture and limited parameters, especially under challenging adverse environments, such as heavy rain and heavy snow. As shown in Fig. 1, comparative results of captured video frames have been presented under normal and adverse environments. We observe that, despite similar objects being in nearly identical positions, they cannot be detected under adverse conditions (circled by the red line at Row 2), leading to a large accuracy drop of 20%, whereas they can be accurately identified under favorable environments using the same lightweight model.

Multiple factors can cause adverse environments, such as poor lighting conditions and extreme weather [13, 38]. In these cases, the video imaging condition significantly deteriorates, manifesting in three distinct differences in the captured frames: increased noise, changed brightness, and blurred objects. All of them can impair the feature extraction of objects, making inference more difficult. To enhance the performance of lightweight models in non-ideal environments on end devices while maintaining real-time inference,

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MM '23, October 29-November 3, 2023, Ottawa, ON, Canada
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0108-5/23/10…$15.00
https://doi.org/10.1145/3581783.3612585

continuous learning has emerged as a promising technique [20, 22, 29]. This approach continuously retrains the lightweight model, or the student model on new data samples, leveraging the knowledge transferred from a more advanced DNN, which is referred to as the teacher model. The new data samples involve specific information relevant to the current environment, allowing the student model to specialize in the given scenario. By retraining the student model with new data samples, it can adapt to changes in the environment, therefore improving overall performance.

However, the retraining process is computationally intensive. It also requires groundtruth labels from a large teacher model, rendering it infeasible to be efficiently executed on resource-constrained end devices [24, 28, 33, 55, 57]. To address this challenge, in this paper, we utilize an edge server to assist with retraining tasks. This is because edge computing can help to meet the rigorous demands of video analytics tasks due to its availability in the proximity of source video cameras [4, 9, 34, 40, 45, 52]. Besides, less transmission bandwidth consumption and lower response latency can be achieved, compared with cloud-based solutions [26, 46, 54, 56]. Instead of offloading all the videos and tasks to the edge server, we aim to perform inference tasks on end cameras and conduct retraining tasks on edge servers, thereby leveraging the computing resources at both sides, as well as meeting bandwidth constraints.

In particular, we propose an edge-assisted real-time video analytics system that aims at enhancing the performance of the student model on end cameras under adverse environments. Once the environmental condition is detected to be changed, the student model on the end cameras is updated with the retrained weight parameters from the edge server. Nonetheless, implementing such a system is non-trivial as it faces three significant challenges. *Firstly*, how to efficiently select data samples for better retraining while the size of data is subject to bandwidth constraints. *Secondly*, how to trigger the retraining timely while avoiding unnecessary retraining process. *Finally*, how to achieve optimal retraining configurations for better retrained model accuracy and faster response. To address those challenges, we first devise an efficient Key Frame Extractor which uses the inference results of the student model to obtain the frames that can best represent the current environments and removes temporal redundancy to meet the current bandwidth constraints. Then, an accuracy-based Trigger Controller is designed to adaptively trigger the retraining process. Moreover, we formulate an optimization problem that aims at selecting the best configurations for retraining tasks to strike a balance between accuracy and total latency. A heuristic algorithm is proposed in the Retraining Manager module to solve the optimization problem. Finally, we develop a test-bed consisting of two end cameras and an edge server to evaluate the performance of the proposed framework. Our main contribution can be summarized as follows.

- We propose a key frame extraction method, which fully utilizes the student inference results to efficiently select the most informative frames for retraining. Temporal redundancy is also removed based on frame difference to meet the bandwidth constraints. This two-step key frame extractor is low-cost and hence can be implemented on end cameras.
- We design a mechanism to adaptively trigger the retraining task based on the accuracy drop of sampled frames, which effectively prevents unnecessary retraining that wastes computation resources on the edge server while maintaining satisfying inference performance.
- We devise a Retraining Manager module to identify the optimal retraining configurations, the resulting algorithm can achieve a balanced tradeoff between inference accuracy and total retraining time overhead.
- We prototype our system on two end devices and one edge server. Then, we validate our approach under three typical adverse environments with the task of object detection. The proposed solution is shown to have up to 24% accuracy improvement and 50% retraining time reduction.

The remainder of this paper is organized as follows. Section 2 describes the background and illustrates relevant experiment that motivates our work. Section 3 presents the details of the system model design. Section 4 demonstrates the evaluation results. Finally, we conclude our paper in Section 5.

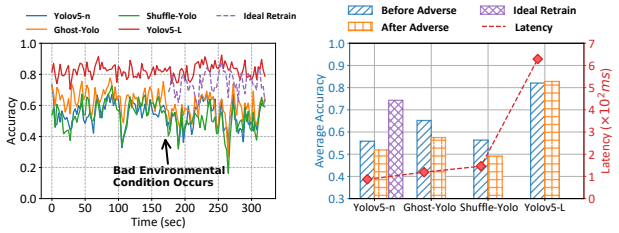
2 BACKGROUND AND MOTIVATION

Recent advances in computer vision have led to the development of state-of-the-art (SOTA) DNNs, which are designed with deeper network layers and wider feature maps, making them capable of covering a wide range of scenes and vision tasks [10, 39, 51]. However, the ever-increasing complexity of DNN architectures also increases the prediction latency and poses a significant challenge to delay-sensitive tasks on resource-constrained end devices [23, 43].

In order to balance the tradeoff between accuracy and latency on resource-constrained end devices, many existing works have exploited the potential of lightweight models with simplified network architecture and reduced amount of parameters, such as GhostNet [11] and ShuffleNet [58], which can deliver remarkable performance in either customized environments or favorable conditions. Nevertheless, although lightweight, the inherent structural characteristics preclude such models from achieving high performance in more challenging scenarios, such as those with dynamic object distributions or adverse environmental conditions. To tackle the first type of challenging scene, continuous learning-based methods have been proposed by many works [2, 21, 32]. Bhardwaj *et al.* have illustrated that the varying object class distributions degrade the efficacy of lightweight models, while retraining is effective in enhancing accuracy under such circumstances [2].

We conduct the following experiments to investigate the ability of SOTA lightweight models in handling the second type of challenging scenarios, *i.e.*, under adverse environments. We compare Yolov5 series models [53], GhostNet [11], and ShuffleNet [58], respectively on NVIDIA Jetson Nano, which is the most widely used yet resource-constrained end devices. We replace the Yolo backbone with GhostNet and ShuffleNet backbones to take advantage of the faster feature extraction capabilities while maintaining the same Yolo detection head. Specifically, we use a 320s video from YouTube[1], during which heavy snow suddenly occurs after the $170 - th$ second and persists for the rest of the time. Additionally, we retrain Yolov5-n with the best retraining configurations (obtained via exhaustive searching) and we disregard any potential increase in latency caused by the retraining process to realize

[1]https://www.youtube.com/watch?v=1EiC9bvVGnk



(a) Accuracy v.s. time curve of different models or methods.

(b) Latency and average accuracy before and after adverse environments.

Figure 2: Comparison of different methods during which adverse environments occur at $t = 170s$.

accuracy-ideal retraining. In this way, we can investigate whether retraining can yield better performance in such scenarios. The F1-Score results over time are depicted in Fig. 2(a), while Fig. 2(b) presents a comparison of average accuracy before and after exposure to adverse environmental conditions, as well as the inference latency per frame. Our key observations are as follows.

- Heavy DNN generates better results with higher latency. Yolov5-L keeps much more stable accuracy all the time and outperforms all lightweight models by a significant margin, even when faced with sudden environmental degradation. However, the heavy computation overhead results in almost six times more latency than other models, making it impractical for deployment on such resource-constrained devices.
- SOTA lightweight models cannot cope with bad environmental conditions. Even the two SOTA lightweight models exhibit a sharp drop in accuracy around $t = 270s$ and suffer obvious average accuracy loss from the poor environment.
- Retraining is promising to enhance performance. The retrained Yolov5-n can yield substantial accuracy benefits under ideal conditions, but it is not feasible to determine the optimal frames and epochs for each retraining instance as the prolonged retraining latency may render the retrained model obsolete for ever-changing video streams.

In summary, continuous retraining is an effective strategy to improve model performance on resource-constrained end devices when dealing with unfavorable environmental conditions. Moreover, it is essential to ensure that each retraining is configured properly to optimize overall accuracy and retraining latency.

3 SYSTEM MODEL

In this section, we present the design of our edge-assisted real-time video analytics system. We start by providing an overview of the system and then discuss its three key components respectively: Key Frame Extractor, Trigger Controller, and Retraining Manager.

3.1 System Overview

As shown in Fig. 3, our video analytics system consists of modules on the end camera and the edge server.

End camera side. A lightweight student model is deployed on the end camera to perform real-time inference, with adaptive updates to accommodate the ever-changing environmental conditions. We develop a Key Frame Extractor that leverages the inference results of the student model to identify frames that can best represent the current environmental conditions within a given time window. Besides, we remove the temporal redundancy via frame differencing to meet the bandwidth requirements, as we offload the retraining task to the edge server. Then, these selected frames and the corresponding inference results of the student model are transmitted to the edge server for further analysis.

Edge server side. After transmitting the data of a window to the edge server, a sophisticated Oracle model generates groundtruth labels for these frames. A Trigger Controller is designed to make adaptive decisions of whether initiate retraining based on the empirical accuracy variation. If retraining is considered to be unnecessary, the received data is stored in the Buffer and used as training samples for the next retraining process. Otherwise, the Retraining Manager leverages the data of both the present window and buffered key frames to obtain a sub-optimal configuration for balancing accuracy and retraining time overhead. Besides, we incorporate the knowledge distillation technique into our retraining process, given its widely accepted role in enabling small models to acquire new complex knowledge quickly [30, 47]. As the selection of different teacher models can impact both accuracy and time cost, we also regard it as a tunable parameter in Retraining Manager, in addition to the epoch number and frame amount. Finally, after the retraining process is done, we update the current model on the end camera with the retrained model to achieve more accurate inference results.

3.2 Key Frame Extractor

Performing retraining task demands substantial computational power and cannot be carried out on end cameras with limited resources which result in high latency [15, 49, 50]. To address this issue, we leverage an edge server to assist in retraining, which entails transmitting data samples from end camera to edge server.

Video frames can offer a plethora of information, but not all of them necessarily contribute to enhancing the performance of model retraining. The reasons are two-fold. First, during the training process, the model parameters are updated only when there are discrepancies between the predicted labels and the groundtruth labels. As a result, frames that the student model can accurately predict do not provide sufficient helpful information for retraining. Second, consecutive frames in the video sequence contain redundant information that does not offer extra gain to the training process. Besides, aggressively sending all of the frames to the edge server for selection is bandwidth inefficient. As a result, conducting key frame extraction directly on device is imperative.

We divide the video stream into windows of equal size and conduct key frame extraction within each window once the detection of the current window is completed. By leveraging the confidence score of student inference results, we can assess if the current student model can accurately predict the current frames, as lower confidence scores suggest higher inference difficulty, indicating possible adverse environments. Moreover, we adopt a frame differencing approach to reduce the temporal redundancy, which enables us to meet the stringent bandwidth requirements in unfavorable network conditions. The frame differencing method is a low-cost technique that allows for the detection of pixel-level changes in brightness, color, and texture within video sequences [42].

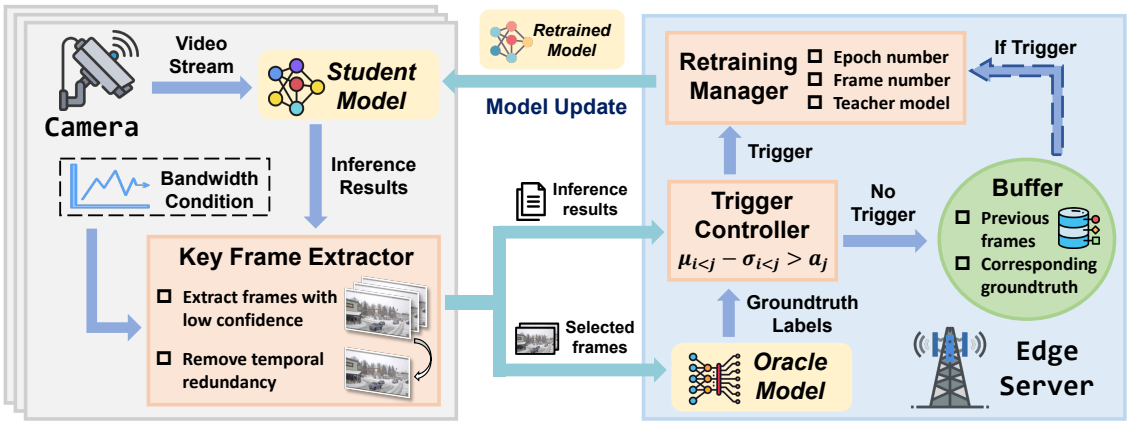


Figure 3: An overview of system design.

Algorithm 1 summarizes the process of our two-step Key Frame Extractor within a time window. *Firstly*, it iterates over each frame in the window and calculates the low-confidence ratio, defined as the proportion of detected objects with low confidence. If the ratio exceeds the pre-defined threshold β, indicating that the majority of objects in the frame are difficult for prediction, this frame is then added to the selected list $\mathcal{S}$. *Next*, we determine the maximum number of frames C that can be transmitted under the current uplink bandwidth limit B_{up}. If the number of frames in $\mathcal{S}$ is less than C, we transmit the whole list directly to the edge server. Otherwise, we compute the frame difference for each frame in $\mathcal{S}$. Since the list $\mathcal{S}$ is only a subset of $\mathcal{F}$ after Step 1, the calculations can be cut down dramatically. As a result, we select top-C frames with the largest frame difference as the most representative frames for offloading.

This low-cost yet effective approach allows us to identify the most informative frames while adhering to bandwidth requirements. In addition, along with the selected frames, the student inference results will also be transmitted to the edge server for further analysis. The data size of this supplementary information is negligible.

3.3 Trigger Controller

Previous works such as AMS [21] and Ekya [2] adopt a fixed update interval. Nonetheless, a large interval may fail to respond timely to sudden changes in the current environment, while a low update interval may cause unnecessary frequent retraining that wastes computational resources on the edge server. Other study like JIT [32] triggers retraining once detected the accuracy of the student model falls below a pre-defined threshold. However, a fixed threshold may not be suitable for all scenarios with varying levels of difficulty. Based on this idea, we devise a subtle trigger method to make the threshold adaptive, realizing both prompt response to environmental changes and efficient use of resources.

After the selected frames and corresponding student inference results of a window are transmitted to the edge server, an advanced Oracle model generates groundtruth labels for these frames. The groundtruth labels are then used to retrain the model and evaluate the accuracy of those selected frames within the current window. By jointly taking into account the accuracy results of each received window, we develop an adaptive Trigger Controller.

Algorithm 1: Key Frame Extraction Algorithm

Input: $\mathcal{F} = \{f_1, f_2, ..., f_i, ...\}$: frame set of current window, β: filtering threshold, B_{up}: current uplink bandwidth.
Output: $\widetilde{\mathcal{S}}$: Low confidence frame set, $\mathcal{S}$: List of selected frames for transmission.

Step1: Extract Low-confidence Frames
for f_i *in* $\mathcal{F}$ **do**
  $L_i \leftarrow$ getLowConfRatio(f_i)
  if $L_i > \beta$ **then**
    $\mathcal{S}.append(f_i)$
$\widetilde{\mathcal{S}} = \mathcal{S}$ ▷ the set of low confidence frames
Step2: Remove Temporal Redundancy
$C \leftarrow$ determine the maximum number of transmitted frames limited by the current available uplink bandwidth B_{up}
if $|\mathcal{S}| \leq C$ **then**
  return $\mathcal{S}$
else
  for f_i *in* $\mathcal{S}$ **do**
    $d_i \leftarrow$ getFrameDifference(f_i)
  $\mathcal{S}^* \leftarrow$ sort $\mathcal{S}$ by d_i with a descending order
  $\mathcal{S} = \mathcal{S}^*[:C]$ ▷ select top-C frames with the largest frame difference
  return $\mathcal{S}$

For each retraining cycle, let $\mathcal{W} = \{W^1, \ldots, W^i, \ldots, W^j\}$ denote all the windows received by the edge server up to the current window W^j. Correspondingly, we define the average accuracy of these windows as $\mathbf{a} = \{a_1, \ldots, a_i, \ldots, a_j\}$. Our trigger mechanism is designed to identify when the performance of the current window is worse than that of previous windows.

As more recently received windows provide more reliable information about the current environment, we adjust the importance of previous windows $\mathcal{W}^{i<j}$ with an exponential decay function. As a result, the weighted average accuracy $\mu_{i<j}$ of previous windows $\mathcal{W}^{i<j}$ can be represented as $\mu_{i<j} = \sum_{i=1}^{j-1} \lambda_i a_i$, where λ_i is the

normalized weight derived from the exponential decay function that gives more importance to newer windows.

Meanwhile, given the inherent limitations of the student model, fluctuations in accuracy are a common occurrence, which can result in unnecessary triggers. To mitigate this issue, we incorporate a reasonable range of accuracy fluctuations for current window, determined by the standard deviation of previous windows $\sigma_{i<j}$. In summary, we trigger retraining if the following equation is satisfied:

$$\mu_{i<j} - \sigma_{i<j} > a_j. \tag{1}$$

Our trigger mechanism ensures that retraining is adaptive to various scenarios and is only initiated when necessary. In cases where the student model performs well and does not require retraining, we store the selected frames and groundtruth labels of this window in the Buffer until next trigger. This allows us to use all the data from the received windows for next retraining. Additionally, the Buffer is cleared after each retraining cycle.

3.4 Retraining Manager

Upon receiving the signal from Trigger Controller, the Retraining Manager identifies the proper configurations to retrain with the data from both the current window and the Buffer, aiming to achieve high accuracy of the retrained model while minimizing the total retraining time cost to prevent the model from being obsolete. Although accuracy-optimal configurations can enhance performance, they also result in a higher time cost, as illustrated in Fig. 4. Additionally, we observe that there is only a marginal improvement in accuracy when the time cost is significantly higher, which necessitates the selection of the best configurations.

We specifically focus on three retraining knobs that play critical roles in both accuracy and time cost: epoch number e, frame amount n, and teacher model used for knowledge distillation m. The k-th retraining configurations can be represented as a collection of these factors $\mathbf{c_k} = (e_k, n_k, m_k)$. Then we devise a utility function for each retraining task to evaluate the trade-off of different configurations between accuracy and time overhead, which is defined as:

$$U_k = A_k - \eta_k T_k, \quad \forall k \in K. \tag{2}$$

where A_k represents the accuracy of k-th retrained model and is related to different configurations of $\mathbf{c_k}$. T_k denotes the total time cost for k-th retraining and η_k profiles the current urgency degree.

Latency Model. The total time overhead T_k for k-th retraining can be divided into four parts: *Part. (1)* data transmission latency, *Part. (2)* groundtruth label generation latency, *Part. (3)* training processing latency and *Part. (4)* model transmission latency. We ignore the time cost for Retraining Manager to generate results as we will demonstrate in Section 4 that this cost can be neglected with our proposed algorithm. Besides, as we have stored data of previous windows in the Buffer, we ignore *Part. (1)* and *Part. (2)* latency from these windows and only consider these two parts latency caused by the trigger window W_k^Γ. Hence, the total latency consisting of four parts is modeled as:

$$T_k = \frac{d_0|\mathcal{S}_k^\Gamma|}{B_{up}} + t_0|\mathcal{S}_k^\Gamma| + e_k n_k t(m_k) + \frac{M_0}{B_{down}}, \quad \forall k \in K. \tag{3}$$

where d_0 and t_0 are two constants representing the data size and the groundtruth label generation time for each frame, respectively. M_0 is the size of the student model. $|\mathcal{S}_k^\Gamma|$ denotes the number of frames in the trigger window W_k^Γ. B_{up}, B_{down} represent the available uplink and downlink bandwidth during transmission, respectively. The *Part. (3)* training processing latency is modeled as $e_k n_k t(m_k)$, since we find a significant linear correlation between $e_k \cdot n_k$ and the training processing latency, as shown in Fig. 5. Here, $t(m_k)$ represents the slope value which varies with different teacher models.

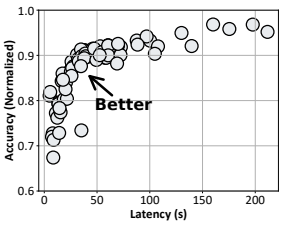


Figure 4: Accuracy and latency performance of different configurations.

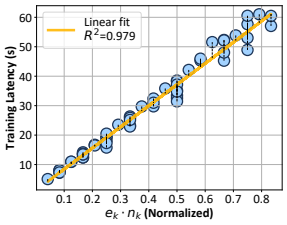


Figure 5: $e_k \cdot n_k$ is approximately linear to the retraining processing latency.

Urgency Degree. The accuracy of student model can be affected differently by various environmental conditions as minor changes can result in a slight decrease, while severe changes can lead to a significant drop, which calls for a faster retraining. As in Step 1 of Algorithm 1, we have obtained the low confidence frame set $\widetilde{\mathcal{S}}$ for each window, a larger number of frames in this set indicates a more challenging environment for the student model. Therefore, we utilize the ratio of low confidence frames in the trigger window W_k^Γ to represent the urgency degree (η_k) for k-th retraining. A higher value of η_k indicates a greater urgency to retrain the current student model. The urgency degree is defined as:

$$\eta_k = \frac{|\widetilde{\mathcal{S}_k^\Gamma}|}{|\mathcal{F}|}, \quad \forall k \in K. \tag{4}$$

This definition of urgency degree can help us to make a better tradeoff between accuracy and latency for each retraining.

Problem Formulation. Our objective is to identify the configuration combination of epoch number, frame amount, and teacher model so as to maximize the utility function for all the retraining processes. This problem can be formally formulated as:

$$\mathcal{P}: \max_{\{\mathbf{e},\mathbf{n},\mathbf{m}\}} \sum_{k=0}^{K} U_k \tag{5}$$

$$\text{s.t.} \quad T_k \le \tau_0, \quad \forall k \in K \tag{6}$$

$$e_k \le e_{max}, n_k \le |\mathcal{S}_k|, \quad \forall k \in K \tag{7}$$

$$m_k \in \mathcal{M}, \quad \forall k \in K \tag{8}$$

where Constraint (6) limits the total time cost to a threshold τ_0, ensuring that the retrained model does not become outdated. Besides, as specified in Constraint (7), the current retraining process must abide by the constraints on the maximum optional frame amount $|\mathcal{S}_k|$ and maximum epoch number e_{max}. Constraint (8) ensures the selection of the teacher model from the given model set $\mathcal{M}$.

Due to this problem is a mixed-integer linear programming (MILP) which is known to be NP-hard, a brute-force approach to solving the problem has a computational complexity of $O(K \cdot |\mathcal{M}| \cdot$

Algorithm 2: Configuration Selection Algorithm

Input: $|\mathcal{S}_k^\Gamma|, |\mathcal{S}_k|, \eta_k$
Output: $\mathbf{c}^* = [e^*, n^*, m^*]$: output solution.

1 $i \leftarrow 0, j \leftarrow 0$ ▷ initialize iteration counter and fail counter
2 $\mathbf{c}^* \leftarrow \mathbf{c_0}$ ▷ initialize output solution
3 $U(\mathbf{c}^*) \leftarrow$getUtility$(\mathbf{c}^*; \eta_k; |\mathcal{S}_k^\Gamma|)$ ▷ compute initial utility score based on Eq.(2)(3)
4 **while** $i < max_iter$ *and* $j < max_fail$ **do**
5 $\quad \mathbf{c}' \leftarrow$ generate a new candidate solution randomly within Constraint (6)(7)(8)
6 $\quad U(\mathbf{c}') \leftarrow$getUtility$(\mathbf{c}'; \eta_k; |\mathcal{S}_k^\Gamma|)$
7 $\quad \Delta f \leftarrow U(\mathbf{c}') - U(\mathbf{c}^*)$ ▷ compute difference of last solution and new solution
8 $\quad$ **if** $\Delta f > 0$ *or* $\exp(\Delta f / U(\mathbf{c}')) > rand(0,1)$ **then**
9 $\quad\quad \mathbf{c}^* \leftarrow \mathbf{c}'$ ▷ accept new solution
10 $\quad\quad j \leftarrow 0$ ▷ reset fail counter
11 $\quad$ **else**
12 $\quad\quad j \leftarrow j + 1$ ▷ update fail counter
13 $\quad i \leftarrow i + 1$ ▷ update iteration counter
14 **return** $\mathbf{c}^*$ ▷ return output solution

$|\mathcal{S}_k| \cdot e_{max}$), making it infeasible to be implemented in practice. Thus, an efficient algorithm should be developed.

Algorithm Design. Instead of considering all retraining tasks simultaneously, we propose Algorithm 2 to identify a sub-optimal configuration for each retraining task individually. At the beginning of the algorithm, we gradually explore the solution space by perturbing the current solution within given constraints. Specifically, each new candidate solution $\mathbf{c}'$ (Line 5) is generated as:

$$e' = \max(\min(e \pm randint(0, e_{min}), e_{max}), e_{min}) \quad (9)$$

$$n' = \max(\min(n \pm randint(0, n_{min}), |\mathcal{S}_k|), n_{min}) \quad (10)$$

$$m' = randchoice(\mathcal{M}) \quad (11)$$

We adopt a new solution c' to update the current optimal solution c^* when its utility score surpasses that of c^*. In case the utility score of c' is inferior, we may still adopt it with a probability according to Metropolis Criterion [31]. We end the search process early if a new candidate solution is not accepted after max_fail times of attempts. Otherwise, we complete all iterations before exiting. Additionally, with the output number of n^*, we select the most recent frames as the training set because these frames are better indicators of current environmental conditions.

In summary, through the collaborative efforts among the Key Frame Extractor on end cameras and the Trigger Controller, the Retraining Manager on edge servers, our system can enhance video analytics accuracy under adverse environmental conditions.

4 PERFORMANCE EVALUATION

In this section, we demonstrate the efficiency of our system design with real-world implementations and experiments on object detection, a fundamental task in many high-level vision applications.

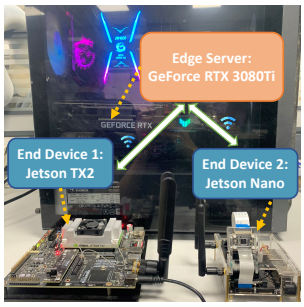


Figure 6: A snapshot of the system prototype implementation.

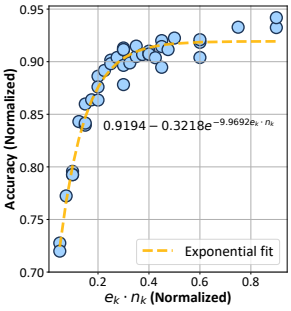


Figure 7: $e_k \cdot n_k$ is fit to accuracy with an exponential function.

4.1 Experimental Settings

Setup. We utilize Yolov5 series models [53] pre-trained on COCO dataset [27] in our evaluation. We develop a test-bed, consisting of two end devices with different computation resources, *i.e.*, Jetson Nano [36] and Jetson TX2 [35]. Both end devices are connected to the same edge server via wireless links. On the end devices, we implement Yolov5-n as the student model with TensorRT engine [44] to accelerate inference. A desktop computer equipped with NVIDIA GeForce RTX 3080Ti GPU acts as the edge server, on which the Oracle model Yolov5-X is deployed to conduct retraining tasks. A snapshot of our system prototype is shown in Fig. 6.

Datasets. We evaluate our design under three typical challenging environmental conditions: Low Light, Heavy Snow, and Heavy Rain. These adverse conditions exhibit varying levels of change intensity, with Low Light scene changing most smoothly and Heavy Rain scene changing most dramatically. Due to the limited availability of datasets on various adverse environments, particularly those containing long-time-span videos with ample valid objects, we collect these videos from YouTube[2], with a resolution of 1920 × 1080 and a frame rate of 30 frames per second.

Parameters. We set the parameters in our proposed algorithms and components, including $\beta = 0.4$, $W_{length} = 10s$, $\tau_0 = 60s$, $n_{min} = 50$, $e_{min} = 5$ and $e_{max} = 100$. Our teacher model set $\mathcal{M}$ comprises of three models, namely Yolov5-s, Yolov5-m, and Yolov5-l. By conducting offline profiling, we are able to not only determine the value of $t(m)$ for each teacher model but also establish the relationship between the accuracy and $e_k \cdot n_k$ across three teacher models. An example of the fitted accuracy curve using Yolov5-s as the teacher model is presented in Fig. 7.

Schemes. We compare with the following schemes.

- **No Retraining (NR).** We run the pre-trained student model directly on end cameras without any specific information from the current video contents.
- **One-Time Retraining (OTR).** We retrain the student model only once with the key frames extracted from the first 60s of the video and test on the rest. This strategy can be seen as our system excluding the component of Trigger Controller.

[2]https://www.youtube.com/watch?v=1EiC9bvVGnk and https://www.youtube.com/watch?v=vSq3eUasKWA

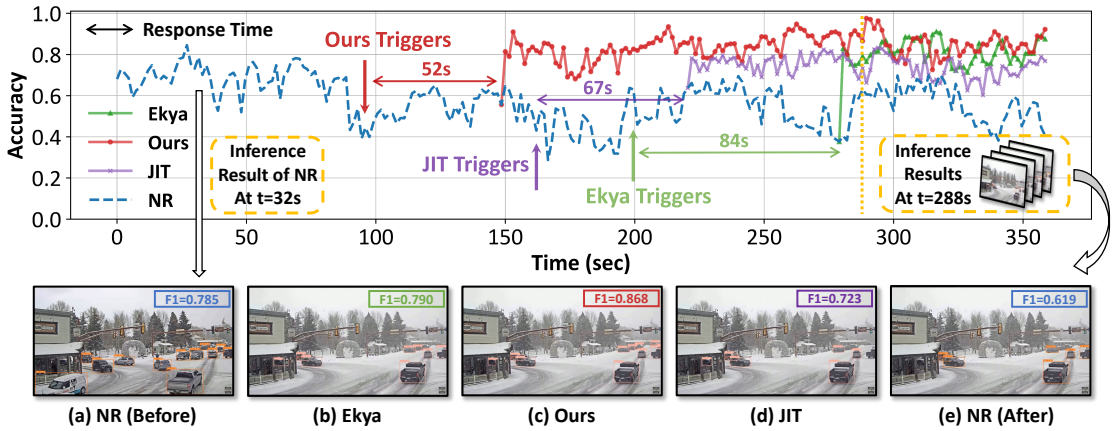


Figure 8: Comparison of the first model update process of different schemes under the Heavy Snow scene.

- **JIT [32].** JIT employs a fixed accuracy threshold to control retraining and uses a retraining strategy where the student model stops retraining when either the accuracy threshold is reached or the maximum iterations have been executed.
- **Ekya [2].** Ekya uses a fixed retraining window of 200s, *i.e.*, updates the student model periodically. Ekya selects the best configurations for retraining through an online micro-profiler, which runs a few extra epochs on a small fraction of training data to build a profile of the training curve for each configuration. In Ekya, both inference and retraining are done on the edge server, indicating that the whole video stream will be continuously offloaded from the camera to the edge server for real-time inference purpose.

4.2 End-to-End Performance

Performance of first model update. We compare the first model update process of different schemes under Heavy Snow conditions in Fig. 8. Before the occurrence of adverse environments, the student model is not retrained but performs well as the inference result of NR at $t = 32s$ shows in Fig. 8(a). Then the heavy snow happens and we compare the detection results of four schemes on the same frame extracted under bad environments at $t = 295s$ in Fig. 8(b)-8(e) respectively. As presented in Fig. 8(e), the previous student model fails to adapt to the degraded environmental conditions with a large accuracy drop. Instead, our adaptive Trigger Controller makes our system the first to detect environmental deterioration and initiate retraining. Besides, our efficient configuration selection algorithm enables our system to retrain and update the student model in just 52s, resulting in a quicker response time and higher overall accuracy. In contrast, JIT and Ekya trigger retraining later than ours and take 67s and 84s to complete the whole retraining process respectively, resulting in a longer response time during which the accuracy maintains low.

Overall accuracy. The average F1-Score results and corresponding standard deviation values under three different scenes are presented in Fig. 9. Our system outperforms all the other methods in terms of accuracy, achieving up to 24% improvements compared to the naive NR strategy. Besides, our system demonstrates a low incidence of sudden accuracy drop due to our timely trigger of retraining when accuracy falls below the expected level. As a result, our method exhibits the lowest standard deviation values across all scenarios, indicating the robustness of our system.

OTR can significantly improve accuracy under Low Light scene, which has a relatively smooth and gradual change in environmental conditions. However, in more volatile environments like Heavy Rain, this approach is less effective because the student model cannot adapt to new changes in environments. JIT triggers retraining when the accuracy threshold is reached, but this fixed threshold is not optimized for different environmental conditions, preventing JIT from gaining better accuracy in more challenging scenes.

Ekya is able to select appropriate configurations for retraining with the help of an online micro-profiler, which works well under less challenging scenes like Low Light. Nevertheless, when confronted with the worse Heavy Rain scene, the micro-profiler fails to profile the training curve correctly due to the difficulty of achieving convergence within just a few epochs, which ultimately leads to inferior output configurations. Besides, the fixed update interval also prevents Ekya from responding to environmental deterioration timely. Compared to Ekya, our method achieves 12% higher accuracy and maintains good performance in all challenging scenes.

Accuracy-latency tradeoff. By jointly considering accuracy and total retraining time cost, we demonstrate our system is capable of striking a balance between both metrics, as shown in Fig. 10. We achieve better accuracy and latency tradeoff than Ekya and JIT across all three scenes due to our efficient configuration selection algorithm, which completes in just 20ms. In contrast, JIT and Ekya take more time to retrain but do not achieve better accuracy due to the deficiency of their retraining strategies. For example, JIT aggressively retrains the model to the pre-defined accuracy level, while Ekya requires additional micro-training to obtain a training curve for configuration selection, both of which have an inferior balance of accuracy and time cost. Besides, longer retraining time also increases the risk of the retrained model becoming outdated, leading to poor performance as the video content changes.

Impact of bandwidth condition. We illustrate our system is robust to bandwidth variations in Fig. 11 under the most challenging Heavy Rain scene. Despite unfavorable bandwidth conditions, our

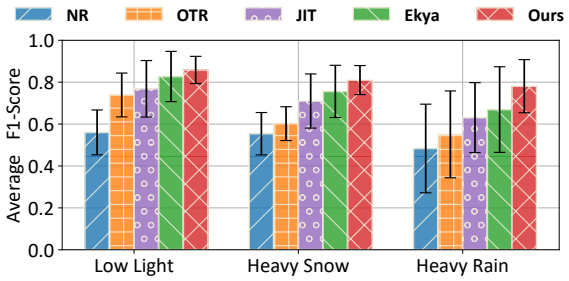


Figure 9: Average F1-Score and standard deviation.

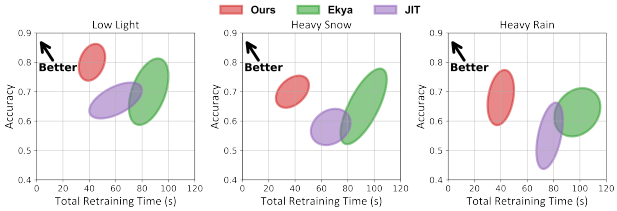


Figure 10: Accuracy v.s. the total retraining time overhead.

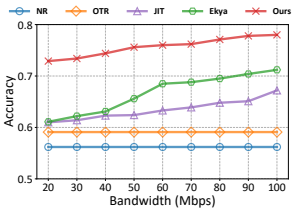


Figure 11: Impact of bandwidth condition.

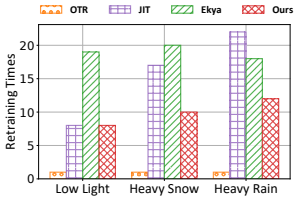


Figure 12: Comparison of retraining times.



Figure 13: Decomposition of total retraining time overhead.

system only experiences a 5% decrease in accuracy. This is because our Key Frame Extractor can select the most representative frames for retraining within current bandwidth constraints. NR and OTR are not sensitive to bandwidth conditions because of none or few communication with the edge. JIT transmits recent frames to the edge with different strides, but it does not consider the transmission under limited bandwidth conditions, leading to accuracy degradation. As Ekya is designed to perform both inference and retraining tasks on the edge server without any video-to-edge offloading specification, we assume it adopts the typical naive offloading strategy, which continuously transmits a small video slot from camera to edge. The result shows that this strategy is most vulnerable to bandwidth variations, with up to 10% accuracy degradation.

4.3 Deep Dive of Retraining Performance

Retraining times. As shown in Fig. 12, our method can avoid unnecessary retraining effectively across all three scenes. JIT employs a fixed accuracy threshold for model updates, which can cause frequent triggers in challenging scenes like Heavy Rain, where accuracy tends to suffer. Ekya adopts a fixed retraining window, which leads to nearly twice the number of retraining times compared to our approach under Low Light Scene. The reduction in retraining times is highly meaningful, as fewer retraining can significantly save the computational resources on the edge server.

Decomposition of total retraining time cost. As mentioned in Section 3.4, our total latency for each retraining process consists of four parts: transmission, labeling, training, and updating. Our proposed algorithm enables us to select configurations achieving both high accuracy and low time cost. A lower retraining time cost can prevent the retrained model from becoming obsolete and save computing resources in the meantime. JIT aggressively retrains the model until the expected accuracy or max iterations, resulting in an unstable time cost. Ekya adopts an online micro-profiler, which brings extra time costs. Fig. 13 compares different parts of the total retraining time cost. We cut the retraining time by 52.7% compared to Ekya while still achieving superior performance. Besides, our time cost for the four parts accounts for 7.8%, 5.2%, 86.2%, and 0.8% respectively, with the three non-training parts (*i.e.*, transmission, labeling, and updating) occupying a relatively small fraction. However, Ekya's micro-profiler takes up 27% of the total runtime (*i.e.*, 25s), hindering it from promptly adapting to new variations in environmental conditions.

5 CONCLUSION

In this paper, we have proposed an edge-assisted approach to continuously update lightweight models on resource-constrained end cameras, aiming to improve the performance of real-time video analytics under adverse environmental conditions. We have devised a Key Frame Extractor to select the most informative frames under bandwidth constraints, an adaptive Trigger Controller to inform model updates promptly, and a Retraining Manager to select configurations for efficient retraining. We have implemented a test-bed and experimental results demonstrated that our system achieves better accuracy under all tested adverse environments with time cost much less than current benchmarks. For the future work, we will focus on further reducing bandwidth consumption by incorporating the encoding process.

ACKNOWLEDGMENTS

This work is supported in part by the Natural Science Foundation of China under Grant 62001180, in part by the Young Elite Scientists Sponsorship Program by CAST under Grant 2022QNRC001, in part by the Natural Science Foundation of Hubei Province of China under Grant 2021CFB338, and in part by the Fundamental Research Funds for the Central Universities, HUST, under Grant 2021XXJS014.

REFERENCES

[1] Ganesh Ananthanarayanan, Paramvir Bahl, Peter Bodík, Krishna Chintalapudi, Matthai Philipose, Lenin Ravindranath, and Sudipta Sinha. 2017. Real-time video analytics: The killer app for edge computing. *computer* 50, 10 (2017), 58–67.

[2] Romil Bhardwaj, Zhengxu Xia, Ganesh Ananthanarayanan, Junchen Jiang, Yuanchao Shu, Nikolaos Karianakis, Kevin Hsieh, Paramvir Bahl, and Ion Stoica. 2022. Ekya: Continuous learning of video analytics models on edge compute servers. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 119–135.

[3] Christopher Canel, Thomas Kim, Giulio Zhou, Conglong Li, Hyeontaek Lim, David G Andersen, Michael Kaminsky, and Subramanya Dulloor. 2019. Scaling video analytics on constrained edge nodes. *Proceedings of Machine Learning and Systems* 1 (2019), 406–417.

[4] Jiasi Chen and Xukan Ran. 2019. Deep learning with edge computing: A review. *Proc. IEEE* 107, 8 (2019), 1655–1674.

[5] Tiffany Yu-Han Chen, Lenin Ravindranath, Shuo Deng, Paramvir Bahl, and Hari Balakrishnan. 2015. Glimpse: Continuous, real-time object recognition on mobile devices. In *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems*. 155–168.

[6] Xu Chen, Chenqiang Gao, Feng Yang, Xiaohan Wang, Yi Yang, and Yahong Han. 2021. Video-to-Image casting: A flatting method for video analysis. In *Proceedings of the 29th ACM International Conference on Multimedia*. 4958–4966.

[7] Yan Cheng, Peng Yang, Ning Zhang, and Jiawei Hou. 2023. Edge-Assisted Lightweight Region-of-Interest Extraction and Transmission for Vehicle Perception. In *GLOBECOM 2023-2023 IEEE Global Communications Conference*. IEEE, to appear.

[8] Xiangxiang Dai, Peng Yang, Xinyu Zhang, Zhewei Dai, and Li Yu. 2022. RESPIRE: Reducing Spatial–Temporal Redundancy for Efficient Edge-Based Industrial Video Analytics. *IEEE Transactions on Industrial Informatics* 18, 12 (2022), 9324–9334.

[9] Kuntai Du, Ahsan Pervaiz, Xin Yuan, Aakanksha Chowdhery, Qizheng Zhang, Henry Hoffmann, and Junchen Jiang. 2020. Server-driven video streaming for deep learning inference. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. 557–570.

[10] Hongpeng Guo, Shuochao Yao, Zhe Yang, Qian Zhou, and Klara Nahrstedt. 2021. CrossRoI: Cross-camera region of interest optimization for efficient real time video analytics at scale. In *Proceedings of the 12th ACM Multimedia Systems Conference*. 186–199.

[11] Kai Han, Yunhe Wang, Qi Tian, Jianyuan Guo, Chunjing Xu, and Chang Xu. 2020. Ghostnet: More features from cheap operations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 1580–1589.

[12] Lu He, Qianyu Zhou, Xiangtai Li, Li Niu, Guangliang Cheng, Xiao Li, Wenxuan Liu, Yunhai Tong, Lizhuang Ma, and Liqing Zhang. 2021. End-to-end video object detection with spatial-temporal transformers. In *Proceedings of the 29th ACM International Conference on Multimedia*. 1507–1516.

[13] Yuanyi He, Peng Yang, Tian Qin, and Ning Zhang. 2023. End-Edge Coordinated Joint Encoding and Neural Enhancement for Low-Light Video Analytics. In *GLOBECOM 2023-2023 IEEE Global Communications Conference*. to appear.

[14] Jiawei Hou, Peng Yang, Tian Qin, and Wen Wu. 2023. Edge-Coordinated On-Road Perception for Connected Autonomous Vehicles Using Point Cloud. In *Proceedings of 31st Biennial Symposium on Communications*. to appear.

[15] Chien-Chun Hung, Ganesh Ananthanarayanan, Peter Bodik, Leana Golubchik, Minlan Yu, Paramvir Bahl, and Matthai Philipose. 2018. Videoedge: Processing camera streams using hierarchical clusters. In *2018 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 115–131.

[16] Samvit Jain, Ganesh Ananthanarayanan, Junchen Jiang, Yuanchao Shu, and Joseph Gonzalez. 2019. Scaling video analytics systems to large camera deployments. In *Proceedings of the 20th International Workshop on Mobile Computing Systems and Applications*. 9–14.

[17] Samvit Jain, Xun Zhang, Yuhao Zhou, Ganesh Ananthanarayanan, Junchen Jiang, Yuanchao Shu, Paramvir Bahl, and Joseph Gonzalez. 2020. Spatula: Efficient cross-camera video analytics on large camera networks. In *2020 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 110–124.

[18] Junchen Jiang, Ganesh Ananthanarayanan, Peter Bodik, Siddhartha Sen, and Ion Stoica. 2018. Chameleon: scalable adaptation of video analytics. In *Proceedings of the 2018 conference of the ACM special interest group on data communication*. 253–266.

[19] Daniel Kang, John Emmons, Firas Abuzaid, Peter Bailis, and Matei Zaharia. 2017. Noscope: optimizing neural network queries over video at scale. *arXiv preprint arXiv:1703.02529* (2017).

[20] Mehrdad Khani, Ganesh Ananthanarayanan, Kevin Hsieh, Junchen Jiang, Ravi Netravali, Yuanchao Shu, Mohammad Alizadeh, and Victor Bahl. 2023. {RECL}: Responsive {Resource-Efficient} Continuous Learning for Video Analytics. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 917–932.

[21] Mehrdad Khani, Pouya Hamadanian, Arash Nasr-Esfahany, and Mohammad Alizadeh. 2021. Real-time video inference on edge devices via adaptive model streaming. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4572–4582.

[22] Dawei Li, Serafettin Tasci, Shalini Ghosh, Jingwen Zhu, Junting Zhang, and Larry Heck. 2019. RILOD: Near real-time incremental learning for object detection at the edge. In *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing*. 113–126.

[23] Yiting Li, Haisong Huang, Qingsheng Xie, Liguo Yao, and Qipeng Chen. 2018. Research on a surface defect detection algorithm based on MobileNet-SSD. *Applied Sciences* 8, 9 (2018), 1678.

[24] Yuanqi Li, Arthi Padmanabhan, Pengzhan Zhao, Yufei Wang, Guoqing Harry Xu, and Ravi Netravali. 2020. Reducto: On-camera filtering for resource-efficient real-time video analytics. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. 359–376.

[25] Jie Lin, Peng Yang, Wen Wu, Ning Zhang, Tao Han, and Li Yu. 2021. Edge learning for low-latency video analytics: Query scheduling and resource allocation. In *2021 IEEE 18th International Conference on Mobile Ad Hoc and Smart Systems (MASS)*. IEEE, 252–259.

[26] Jie Lin, Peng Yang, Ning Zhang, Feng Lyu, Xianfu Chen, and Li Yu. 2022. Low-latency edge video analytics for on-road perception of autonomous ground vehicles. *IEEE Transactions on Industrial Informatics* 19, 2 (2022), 1512–1523.

[27] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*. Springer, 740–755.

[28] Shengzhong Liu, Tianshi Wang, Jinyang Li, Dachun Sun, Mani Srivastava, and Tarek Abdelzaher. 2022. Adamask: Enabling machine-centric video streaming with adaptive frame masking for dnn inference offloading. In *Proceedings of the 30th ACM international conference on multimedia*. 3035–3044.

[29] Davide Maltoni and Vincenzo Lomonaco. 2019. Continuous learning in single-incremental-task scenarios. *Neural Networks* 116 (2019), 56–73.

[30] Rakesh Mehta and Cemalettin Ozturk. 2018. Object detection at 200 frames per second. In *Proceedings of the European Conference on Computer Vision (ECCV) Workshops*. 0–0.

[31] Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. 1953. Equation of state calculations by fast computing machines. *The journal of chemical physics* 21, 6 (1953), 1087–1092.

[32] Ravi Teja Mullapudi, Steven Chen, Keyi Zhang, Deva Ramanan, and Kayvon Fatahalian. 2019. Online model distillation for efficient video inference. In *Proceedings of the IEEE/CVF International conference on computer vision*. 3573–3582.

[33] Taslim Murad, Anh Nguyen, and Zhisheng Yan. 2022. DAO: Dynamic Adaptive Offloading for Video Analytics. In *Proceedings of the 30th ACM International Conference on Multimedia*. 3017–3025.

[34] Shadi A Noghabi, Landon Cox, Sharad Agarwal, and Ganesh Ananthanarayanan. 2020. The emerging landscape of edge computing. *GetMobile: Mobile Computing and Communications* 23, 4 (2020), 11–20.

[35] NVIDIA Corporation. 2017. *NVIDIA Jetson TX2 Developer Kit*. Retrieved 2017 from https://www.nvidia.com/en-us/autonomousmachines/embedded-systems/jetson-tx2.

[36] NVIDIA Corporation. 2019. *NVIDIA Jetson Nano Developer Kit*. Retrieved 2019 from https://developer.nvidia.com/embedded/jetson-nano-developer-kit.

[37] Iyiola E Olatunji and Chun-Hung Cheng. 2019. Video analytics for visual surveillance and applications: An overview and survey. *Machine Learning Paradigms: Applications of Learning and Analytics in Intelligent Systems* (2019), 475–515.

[38] Sibendu Paul, Utsav Drolia, Y Charlie Hu, and Srimat T Chakradhar. 2021. Aqua: Analytical quality assessment for optimizing video analytics systems. In *2021 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 135–147.

[39] Zhaobo Qi, Shuhui Wang, Chi Su, Li Su, Weigang Zhang, and Qingming Huang. 2020. Modeling temporal concept receptive field dynamically for untrimmed video analysis. In *Proceedings of the 28th ACM International Conference on Multimedia*. 3798–3806.

[40] Xukan Ran, Haolianz Chen, Xiaodan Zhu, Zhenming Liu, and Jiasi Chen. 2018. Deepdecision: A mobile deep learning framework for edge video analytics. In *IEEE INFOCOM 2018-IEEE conference on computer communications*. IEEE, 1421–1429.

[41] Daniel Rivas, Francesc Guim, Jordà Polo, Pubudu M Silva, Josep Ll Berral, and David Carrera. 2022. Towards automatic model specialization for edge video analytics. *Future Generation Computer Systems* 134 (2022), 399–413.

[42] Nishu Singla. 2014. Motion detection based on frame difference method. *International Journal of Information & Computation Technology* 4, 15 (2014), 1559–1565.

[43] Mingxing Tan and Quoc Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*. PMLR, 6105–6114.

[44] TensorRT 2018. *TensorRT: Inference Accelerator for Precision Inference at Scale*. Retrieved 2018 from https://developer.nvidia.com/tensorrt

[45] Chengzhi Wang, Peng Yang, Jie Lin, Wen Wu, and Ning Zhang. 2022. Object-Based Resolution Selection for Efficient Edge-Assisted Multi-Task Video Analytics. In *GLOBECOM 2022-2022 IEEE Global Communications Conference*. IEEE, 5081–5086.

[46] Can Wang, Sheng Zhang, Yu Chen, Zhuzhong Qian, Jie Wu, and Mingjun Xiao. 2020. Joint configuration adaptation and bandwidth allocation for edge-based

real-time video analytics. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 257–266.

[47] Lin Wang and Kuk-Jin Yoon. 2021. Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks. *IEEE transactions on pattern analysis and machine intelligence* 44, 6 (2021), 3048–3068.

[48] Shangguang Wang, Yan Guo, Ning Zhang, Peng Yang, Ao Zhou, and Xuemin Shen. 2019. Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach. *IEEE Transactions on Mobile Computing* 20, 3 (2019), 939–951.

[49] Yue Wang, Xiaofeng Tao, Xuefei Zhang, Ping Zhang, and Y Thomas Hou. 2019. Cooperative task offloading in three-tier mobile computing networks: An ADMM framework. *IEEE Transactions on Vehicular Technology* 68, 3 (2019), 2763–2776.

[50] Wen Wu, Peng Yang, Weiting Zhang, Conghao Zhou, and Xuemin Shen. 2020. Accuracy-guaranteed collaborative DNN inference in industrial IoT via deep reinforcement learning. *IEEE Transactions on Industrial Informatics* 17, 7 (2020), 4988–4998.

[51] Peng Yang, Jiawei Hou, Li Yu, Wenxiong Chen, and Ye Wu. 2023. Edge-coordinated energy-efficient video analytics for digital twin in 6G. *China Communications* 20, 2 (2023), 14–25.

[52] Peng Yang, Feng Lyu, Wen Wu, Ning Zhang, Li Yu, and Xuemin Sherman Shen. 2019. Edge coordinated query configuration for low-latency and accurate video analytics. *IEEE Transactions on Industrial Informatics* 16, 7 (2019), 4855–4864.

[53] Yolov5 v6.2 2022. *YOLOv5 by Ultralytics*. Retrieved Feb, 2022 from https://github.com/ultralytics/yolov5

[54] Xiao Zeng, Biyi Fang, Haichen Shen, and Mi Zhang. 2020. Distream: scaling live video analytics with workload-adaptive distributed edge intelligence. In *Proceedings of the 18th Conference on Embedded Networked Sensor Systems*. 409–421.

[55] Ben Zhang, Xin Jin, Sylvia Ratnasamy, John Wawrzynek, and Edward A Lee. 2018. Awstream: Adaptive wide-area streaming analytics. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 236–252.

[56] Miao Zhang, Fangxin Wang, and Jiangchuan Liu. 2022. Casva: Configuration-adaptive streaming for live video analytics. In *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2168–2177.

[57] Wuyang Zhang, Zhezhi He, Luyang Liu, Zhenhua Jia, Yunxin Liu, Marco Gruteser, Dipankar Raychaudhuri, and Yanyong Zhang. 2021. Elf: accelerate high-resolution mobile deep vision with content-aware parallel offloading. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*. 201–214.

[58] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. 2018. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 6848–6856.